Trabajo Práctico 7 — Resolución

Sistemas Basados en Conocimiento

Inteligencia Artificial — UNS — 2026

Las fuentes van en notas al pie. **Cátedra**: diapositivas de A. J. García (2026). **AIMA**: Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. (2010). **PMG**: Poole, Mackworth & Goebel, *Computational Intelligence: A Logical Approach* (1998).

Índice

Ejercicio 1 **2**

Ejercicio 2 **4**

Ejercicio 3 **6**

Ejercicio 4 **8**

Ejercicio 5 **10**

Ejercicio 6 **12**

Ejercicio 7 **15**

Ejercicio 8 **23**

Ejercicio 9 **28**

a) 28

Ejercicio 1

Consigna

¿Qué elementos forman un sistema basado en conocimiento (SBC)? ¿Cómo se relacionan entre ellos?

Elementos. La arquitectura básica de un SBC tiene tres módulos:[1]

- **Base de conocimiento (KB)**: el conocimiento del dominio, representado como **reglas** para hacer inferencias y **hechos** (por ejemplo, los datos percibidos).
- **Máquina de inferencia**: usa las reglas de la KB para **sacar conclusiones** a partir de los datos percibidos.
- **Interface**: se comunica con el **entorno**. Muestra las sugerencias, procesa las percepciones, busca información en la web, interpreta comandos de voz, etc. Depende mucho del dominio de aplicación y de los sensores y acciones disponibles para el agente.

Relaciones. Según el diagrama de la arquitectura:[2]

- **Interface ↔ entorno**: por la interface entran las percepciones y consultas y salen las respuestas (conclusiones, recomendaciones).
- **Interface ↔ KB y máquina de inferencia**: la interface lleva a la KB los datos percibidos y le pasa las consultas a la máquina de inferencia, que devuelve las conclusiones.
- **Máquina de inferencia ↔ KB**: la máquina de inferencia razona sobre las reglas y los hechos de la KB. La representación del conocimiento tiene que estar **en completa sintonía** con el mecanismo de inferencia.
- **Diseño modular**: la KB es un módulo independiente del mecanismo de inferencia. Por eso se pueden agregar o quitar reglas y hechos dinámicamente sin cambiar el código del mecanismo de inferencia, y usar la misma máquina de inferencia en otros dominios.[3]

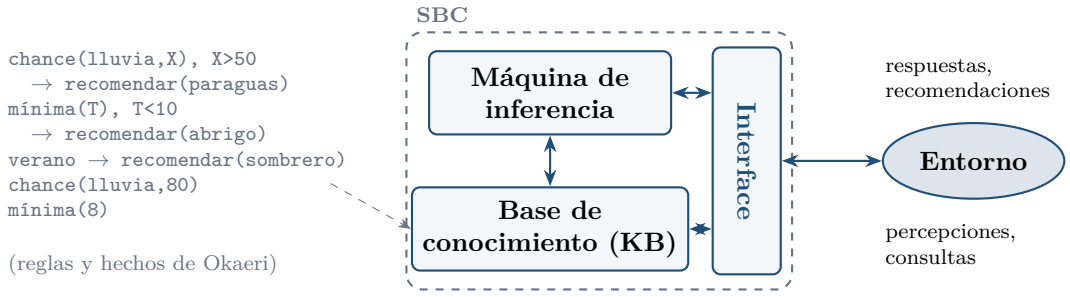


Figura 1: Arquitectura básica de un SBC, con parte de la KB del agente de compañía Okaeri.

[1] Cátedra, Episodio VIII, «Sistema Basado en Conocimiento (SBC)» y «Ejemplo simplificado de SBC del agente de compañía "Okaeri"». Misma arquitectura en PMG, §6.2, p. 200 ("*The main components of the system are the control or inference engine, the knowledge base, and the user interface*").
[2] Cátedra, Episodio VIII, «¿Cómo se implementa un SBC?».
[3] Cátedra, Episodio VIII, «SBC que usa la máquina de inferencia de Prolog».

Elemento	Qué hace	En el agente Okaeri
Base de conocimiento	guarda reglas y hechos del dominio	`chance(lluvia,X), X>50` → `recomendar(paraguas)`; `chance(lluvia,80)`
Máquina de inferencia	saca conclusiones con las reglas de la KB	concluye `recomendar(paraguas)`
Interface	percibe del entorno y muestra los resultados	lee el pronóstico; dice "No te olvides de llevar un paraguas"

EXTRAS

- **Arquitectura de un sistema**: "*modelo conceptual que define la estructura del sistema, indicando (generalmente en forma gráfica) los elementos que lo componen y cómo se relacionan*". Para tener razonamiento automático hacen falta herramientas tanto para representar el conocimiento del dominio como para obtener conclusiones.[4]
- En AIMA, la KB es "*a set of sentences*", cada una expresada en un *knowledge representation language*. Las sentencias que se toman como dadas, sin derivarlas de otras, se llaman **axiomas**. La KB puede arrancar con *background knowledge*.[5]
- PMG distingue cuatro **roles de personas** alrededor de un SBC:[6]
  - *software engineers*: construyen la máquina de inferencia y la interface; no necesitan saber nada del contenido de la KB;
  - *knowledge engineers*: diseñan, construyen y depuran la KB consultando a los expertos;
  - *domain experts*: tienen el conocimiento que se quiere capturar, pero no suelen saber cómo funciona la inferencia;
  - *users*: conocen el caso particular, pero no el dominio. Necesitan una interface simple y natural.
- **Cuestiones a resolver en un SBC**:[7]
  - *derivación de nuevo conocimiento*: aprovechar las conexiones entre piezas de conocimiento para deducir, diagnosticar o encontrar explicaciones;
  - *explicabilidad* (ver ejercicio 5);
  - *información incompleta*: la información del agente en general es parcial;
  - *dinámica de la información*: cambia con el tiempo, o el agente la busca cuando la necesita;
  - *información contradictoria* (ver ejercicio 6).

[4] Cátedra, Episodio VIII, «Sistema Basado en Conocimiento (SBC)».
[5] AIMA, §7.1, p. 235.
[6] PMG, §6.2, p. 200–201.
[7] Cátedra, Episodio VIII, «Cuestiones que hay que abordar en SBC».

Ejercicio 2

Consigna

¿Qué tipo de agente utiliza un SBC? Considerando el ciclo de comportamiento de un agente, ¿en qué partes de este ciclo se utiliza el SBC?

Tipo de agente. Un **agente deliberativo**: tiene que sacar una conclusión, dar una recomendación o tomar una decisión, y para eso usa la información que percibe en el momento y lo que puede inferir por su "propio razonamiento".[8]

Ciclo y uso del SBC. El ciclo del agente deliberativo es:[9]

`REPETIR` percibir → revisar las creencias → razonar sobre el conocimiento → seleccionar un curso de acción → actuar `HASTA el fin`

El SBC interviene en todo el ciclo, cada paso a través de uno de sus módulos:

- **Percibir**: la **interface** procesa las percepciones (p. ej. lee de la página del clima `chance(lluvia,80)`, `mínima(8)`).
- **Revisar las creencias**: lo percibido se incorpora a la **base de conocimiento** como hechos.
- **Razonar sobre el conocimiento**: es el corazón del SBC. La **máquina de inferencia** usa las reglas y hechos de la KB para sacar conclusiones (`lloverá`, `frío`).
- **Seleccionar un curso de acción**: la acción sale de las conclusiones inferidas (`recomendar(paraguas)`, `recomendar(abrigo)`).
- **Actuar**: la **interface** ejecuta la acción en el entorno ("No te olvides de llevar un paraguas").

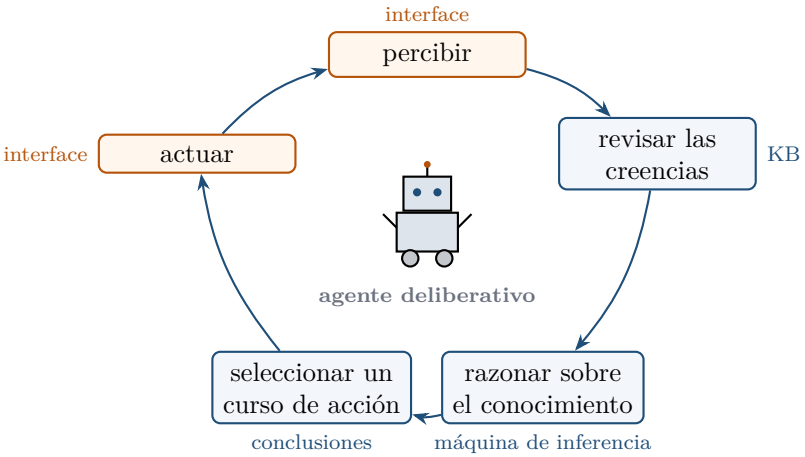


Figura 2: Ciclo del agente deliberativo y módulo del SBC que actúa en cada paso.

[8] Cátedra, Episodio I, parte 2, «¿Por qué un agente deliberativo?». Misma idea en Cátedra, Episodio VIII, «Aplicaciones con razonamiento automático».
[9] Cátedra, Episodio I, parte 2, «Agente deliberativo».

Paso del ciclo	Módulo del SBC	Ejemplo (pronóstico del clima)
percibir	interface	lee `chance(lluvia,80)`, `mínima(8)`
revisar las creencias	base de conocimiento	agrega esos hechos a la KB
razonar sobre el conocimiento	máquina de inferencia	infiere `lloverá`, `frío`
seleccionar un curso de acción	conclusiones de la inferencia	`recomendar(paraguas)`, `recomendar(abrigo)`
actuar	interface	"No te olvides de llevar un paraguas"

EXTRAS

- **Contraste con el agente reactivo**: su ciclo es solo "percibir → actuar reactivamente", con reglas condición–acción (termostato, puerta automática). No mantiene creencias ni razona, así que no usa un SBC.[10]
- **El agente Mer-val** instancia el ciclo así:[11]
  - esperar una consulta;
  - actualizar la información de las empresas con sitios especializados de Internet;
  - hacer inferencias con las reglas de los expertos en mercado bursátil;
  - dar la respuesta con la recomendación.
- **El agente basado en conocimiento de AIMA** hace tres cosas en cada llamada:[12]
  - TELL: le dice a la KB lo que percibe;
  - ASK: le pregunta a la KB qué acción hacer, lo que puede requerir mucho razonamiento;
  - TELL: le dice a la KB qué acción eligió, y la ejecuta.
- Ese agente se puede describir en el **nivel de conocimiento**: alcanza con especificar qué sabe y cuáles son sus objetivos, sin importar cómo está implementado. Construirlo diciéndole (TELL) lo que necesita saber es el **enfoque declarativo**. El enfoque procedural, en cambio, codifica el comportamiento directamente como código.[13]

[10] Cátedra, Episodio I, parte 2, «Agente reactivo».
[11] Cátedra, Episodio VIII, «Agente de recomendación en el Mercado de Valores (Mer-val)».
[12] AIMA, §7.1, p. 235–236.
[13] AIMA, §7.1, p. 236.

Ejercicio 3

Consigna

¿Podrías considerar al intérprete de Prolog como un SBC? ¿Cómo instanciarías cada elemento del SBC?

No por sí solo; sí junto con un programa Prolog que haga de base de conocimiento. Un intérprete de Prolog puede usarse como **máquina de inferencia** ya existente, y para estar en sintonía con él la base de conocimiento tiene que ser un programa Prolog.[14] El intérprete aporta dos de los tres módulos; la KB hay que escribirla para cada dominio:[15]

- **Máquina de inferencia** → el **intérprete de Prolog** (lo aporta el intérprete).
- **Interface** → el **top-level** del intérprete, como interface rudimentaria: se escriben consultas y se obtienen respuestas (lo aporta el intérprete).
- **Base de conocimiento** → un **programa Prolog** con reglas y hechos (**no** lo aporta el intérprete). Sin él, el intérprete no tiene conocimiento del dominio y no puede sacar ninguna conclusión. Los hechos (p. ej. `chance(lluvia,80)`) pueden obtenerse dinámicamente de Internet y estar actualizados.
- **Entorno** → el usuario que consulta y las fuentes (p. ej. sitios web) de donde salen los hechos.

```
recomendar(paraguas) :- chance(lluvia, X), X > 50.
recomendar(abrigo)   :- minima(Temp), Temp < 10.
recomendar(sombrero) :- verano.
verano :- hemisferio(sur), mes(M), member(M,[12,1,2]).
chance(lluvia,80).          % hechos que pueden venir de Internet
minima(8).

?- recomendar(X).           % consulta en el top-level
X = paraguas ;
X = abrigo
```

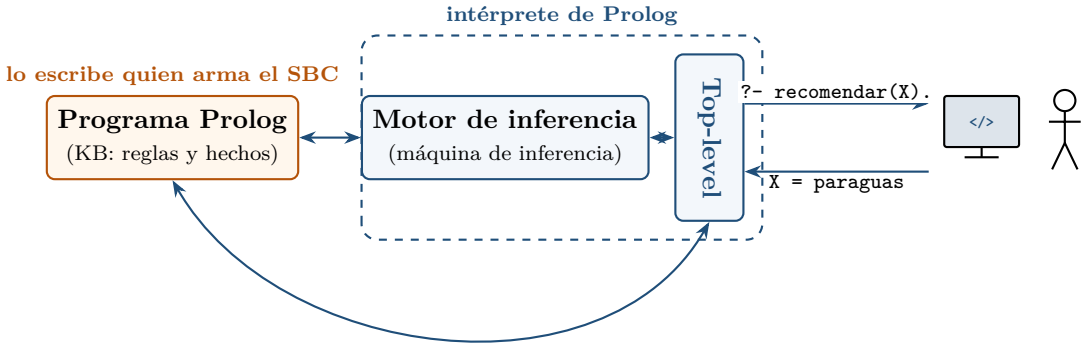


Figura 3: SBC armado con el intérprete de Prolog: el intérprete aporta la máquina de inferencia y el top-level; el programa Prolog (la KB) se agrega aparte.

[14] Cátedra, Episodio VIII, «¿Cómo se implementa un SBC?».
[15] Cátedra, Episodio VIII, «SBC que usa la máquina de inferencia de Prolog».

Elemento del SBC	Instancia en Prolog	¿Lo aporta el intérprete?
Máquina de inferencia	intérprete de Prolog	sí
Interface	top-level (consultas y respuestas)	sí (rudimentaria)
Base de conocimiento	programa Prolog (reglas y hechos)	**no**: hay que escribirlo
Entorno	usuario que consulta; fuentes de los hechos (Internet)	— (externo al SBC)

EXTRAS

- **Límites de Prolog estándar como SBC**:
  - no tiene **negación fuerte** ni un mecanismo para manejar información contradictoria. En el ejemplo Mer-val responde `true` tanto a `comprar_acciones(acme)` como a `no_comprar_acciones(acme)`;[16]
  - solo tiene **negación por falla** (`\+p` tiene éxito cuando la derivación de `p` falla). Sirve para información incompleta, pero solo puede ir en el cuerpo de una regla o en una consulta, nunca en la cabeza ni en un hecho.[17]
- **Extensiones**: con poco esfuerzo se puede armar un "top-level extendido" con una interacción más elaborada, que dé explicaciones o que sea proactivo (sugerencias sin que haya consultas).[18]
- **Prolog en AIMA**:[19]
  - la programación lógica se acerca al ideal declarativo: expresar el conocimiento en un lenguaje formal y resolver problemas corriendo inferencia sobre ese conocimiento ("*Algorithm = Logic + Control*");
  - "*Many expert systems have been written in Prolog for legal, medical, financial, and other domains*";
  - el intérprete ejecuta **encadenamiento hacia atrás en profundidad**, probando las cláusulas en el orden en que están escritas.

[16] Cátedra, Episodio VIII, «Consulta: ¿comprar acciones de acme?».
[17] Cátedra, Episodio VIII, «Negación por falla en Prolog».
[18] Cátedra, Episodio VIII, «SBC que usa la máquina de inferencia de Prolog».
[19] AIMA, §9.4.2, p. 339.

Ejercicio 4

Consigna

Investigue el uso actual de SBC y comente al menos dos ejemplos de aplicaciones que utilicen estos sistemas.

La cátedra muestra ejemplos didácticos (Okaeri, Mer-val), no aplicaciones reales en uso, así que los dos ejemplos salen de otras fuentes.

1. Soporte a la decisión clínica (alertas de medicamentos). [20]

- Las historias clínicas electrónicas avisan al médico, mientras receta, si hay **interacciones entre medicamentos** o alergias del paciente.
- La **KB** tiene reglas sobre combinaciones de medicamentos peligrosas. Los **hechos** son los datos del paciente: lo que ya toma, sus alergias y lo que se le está recetando.
- La **máquina de inferencia** cruza las reglas con los hechos. La **interface** muestra la alerta dentro del flujo de trabajo del médico.
- En EE. UU., el programa de historias clínicas electrónicas de CMS exigía los controles de interacción medicamento–medicamento y medicamento–alergia.
- El estándar HL7 *CDS Hooks* permite que la historia clínica llame a un servicio de soporte a la decisión externo. Así, la lógica de las reglas vive fuera de la historia clínica y se actualiza por separado (la misma modularidad que separa la KB de la máquina de inferencia).

2. Leyes como código (Catala y el impuesto a la renta en Francia). [21]

- **Catala** es un lenguaje para escribir la legislación como código ejecutable. Su estructura imita la de la ley: **reglas generales con excepciones**.
- La **KB** es el texto legal formalizado (las reglas del impuesto o de un beneficio). Los **hechos** son los datos de un contribuyente. La **inferencia** calcula cuánto corresponde pagar o cobrar.
- Se usa en un proyecto con la administración tributaria francesa (DGFiP) para reescribir, a partir de los textos legales, el cálculo del impuesto a la renta.
- Al formalizar los beneficios familiares franceses con Catala se encontró un error en la implementación oficial.

	Alertas clínicas	**Catala (impuestos)**
Base de conocimiento	reglas de interacciones y alergias	leyes formalizadas: reglas generales y excepciones
Hechos	medicamentos y alergias del paciente	datos del contribuyente
Máquina de inferencia	motor de reglas de la historia clínica o servicio externo	compilador o intérprete de Catala
Interface	alerta al médico mientras receta	sistema de cálculo de la administración tributaria

[20] AHRQ, *Individualized Drug Interaction Alerts*, `https://digital.ahrq.gov/ahrq-funded-projects/individualized-drug-interaction-alerts`; CMS, *Clinical Decision Support Rule*, `https://www.cms.gov/Regulations-and-Guidance/Legislation/EHRIncentivePrograms/downloads/11ClinicalDecisionSupportRule.pdf`; HL7, *CDS Hooks*, `https://cds-hooks.hl7.org/`.

[21] Merigoux, Chataing & Protzenko, *Catala: A Programming Language for the Law* (ICFP 2021), `https://arxiv.org/abs/2103.03198`; `https://github.com/catalalang/catala`; Northwestern Law, `https://news.law.northwestern.edu/news/sarah-lawsky-worked-on-a-tax-law-code-that-the-french-government-deemed-officially-awesome/`.

EXTRAS

- **Motores de reglas de negocio** (p. ej. Drools, base de Red Hat Decision Manager): se usan en seguros y bancos, por ejemplo para decidir qué productos ofrecerle a un solicitante de un seguro. Las reglas las escriben los analistas de negocio, separadas del código, y por eso se pueden cambiar seguido.[22]
- **Historia: los sistemas expertos**:[23]
  - **DENDRAL** (1969) infería estructuras moleculares a partir de un espectrómetro de masas. Fue el primer sistema exitoso basado en mucho conocimiento: su capacidad venía de muchas reglas de propósito específico. Los sistemas posteriores separaron limpiamente el conocimiento (las reglas) del componente de razonamiento.
  - **MYCIN** diagnosticaba infecciones de la sangre con unas 450 reglas. Rendía como algunos expertos y bastante mejor que los médicos residentes. Sus reglas se obtuvieron entrevistando expertos e incorporaban la incertidumbre con **factores de certeza** (la estrategia MYCIN del ejercicio 7).
  - **R1** (en DEC) fue el primer sistema experto comercial exitoso. Configuraba pedidos de computadoras y en 1986 le ahorraba a la empresa unos $40 millones por año.
- La regla general con excepciones de Catala es el mismo tipo de conocimiento que las **reglas rebatibles** de DeLP: información tentativa que puede ser derrotada.[24]

[22] Red Hat, *JBoss BRMS 6.0 Release Notes*, «Use Case: Business Decision Management in the Insurance Industry», `https://docs.redhat.com/en/documentation/red_hat_jboss_brms/6.0/html/6.0.0_release_notes/use_cases3`.
[23] AIMA, §1.3.5–1.3.6, p. 22–24.
[24] Cátedra, Episodio X, parte 2, «Defeasible Logic Programing (DeLP)».

Ejercicio 5

Consigna

¿Cómo se imagina que un SBC pueda retornar una explicación de su respuesta (o conclusión o recomendación)?

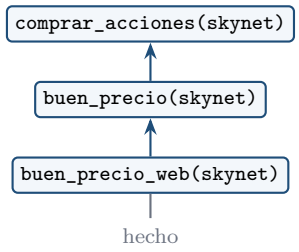
La explicabilidad es una de las cuestiones de un SBC: las razones que llevan a una conclusión o decisión deberían poder auditarse.[25]

- **La derivación es la explicación**: el conjunto de reglas que permite inferir una conclusión dice cómo se llegó a ella y es, además, una razón a favor de esa conclusión.[26]
- **Cómo obtenerla**: extendiendo la **interface** para que, además de la respuesta, devuelva la derivación que hizo la máquina de inferencia. En Prolog, con el predicado predefinido `clause/2`:[27]
  - `deriv(Q,D)` junta en la lista `D` las reglas instanciadas y los hechos usados para derivar `Q`;
  - `explicar(Q)` llama a `deriv` y muestra la lista en pantalla.
- **Con información contradictoria no alcanza con una derivación**: si hay razones a favor y en contra, la explicación detallada es el análisis exhaustivo de todos los argumentos relacionados con la consulta, a favor y en contra.[28] En DeLP, ese análisis queda en los **árboles de dialéctica**, que dan además una explicación de la respuesta.[29]

```
deriv(true, []) :- !.
deriv((Pri,Resto), L) :- !, deriv(Pri,A), deriv(Resto,B), append(A,B,L).
deriv(Cabeza, [(Cabeza :- Cuerpo) | L1]) :- clause(Cabeza, Cuerpo), deriv(Cuerpo, L1).
explicar(Q) :- deriv(Q, D), mostrar(D).

?- explicar(comprar_acciones(skynet)).
comprar_acciones(skynet) :- buen_precio(skynet)
buen_precio(skynet) :- buen_precio_web(skynet)
buen_precio_web(skynet) :- true
```

(a) Una derivación



(b) Argumentos a favor y en contra

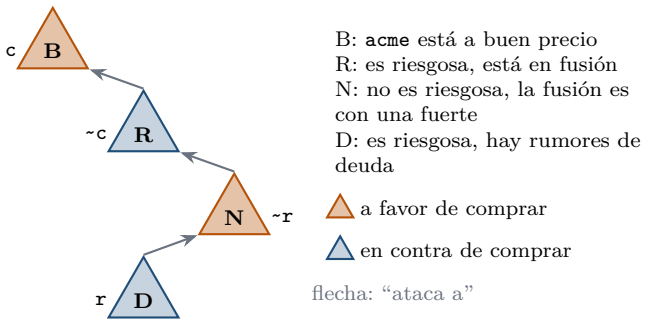


Figura 4: (a) La derivación de `comprar_acciones(skynet)` como explicación. (b) Explicación de "¿por qué no comprar acciones de `acme`?": cada argumento ataca al anterior (c = `comprar_acciones(acme)`, r = `riesgosa(acme)`).

EXTRAS

[25] Cátedra, Episodio VIII, «Cuestiones que hay que abordar en SBC».
[26] Cátedra, Episodio VIII, «Consultando por una recomendación».
[27] Cátedra, Episodio VIII, «Extendiendo la interface».
[28] Cátedra, Episodio X, parte 1, «Brindando una explicación detallada».
[29] Cátedra, Episodio X, parte 2, «Respuesta a una consulta».

- La explicación puede presentarse en lenguaje natural con un top-level extendido. Ejemplo:[30]
  - "Ale: recomendame." — "Oki: paraguas y abrigo."
  - "Ale: explicame." — "Oki: chance de lluvia es 80 %, temperatura mínima de hoy 8 grados."
- **Por qué hace falta explicar**: un sistema no puede dar una respuesta y esperar que el usuario se la crea. Por ejemplo, un médico es legalmente responsable del tratamiento, así que tiene que convencerse de que el diagnóstico es correcto. La misma herramienta sirve para depurar la KB.[31]
- **Preguntas HOW y WHY** (PMG):[32]
  - **HOW**: después de una respuesta, el sistema muestra la instancia de la regla con la que la dedujo. Después se puede preguntar HOW por cada átomo del cuerpo, y así se baja por el **árbol de prueba** (*proof tree*).
  - **WHY**: cuando el sistema le hace una pregunta al usuario, este puede preguntar por qué. El sistema responde con la regla que necesitaba ese dato, y preguntando WHY repetidamente se sube por el árbol hasta la consulta original.
  - Saber por qué se hizo una pregunta aumenta la confianza del usuario. Además, una pregunta irrelevante suele ser síntoma de un problema más profundo en la KB.
- En los **sistemas de mantenimiento de la verdad** (TMS), una explicación de P es un conjunto de sentencias E que implica P. Puede incluir **suposiciones** no probadas (p. ej. "la batería está descargada" para explicar que el auto no arranca). En general se prefieren las explicaciones **mínimas**.[33]

---

[30] Cátedra, Episodio VIII, «SBC que usa la máquina de inferencia de Prolog».
[31] PMG, §6.5, p. 217.
[32] PMG, §6.5, p. 217–221.
[33] AIMA, §12.6.2, p. 462.

Ejercicio 6

Consigna

¿Qué ventajas o desventajas puede tener requerir obligatoriamente que la base de conocimiento sea consistente (libre de contradicciones)? ¿Qué consideraciones debemos tener en cuenta si se permite derivar información contradictoria a partir de la base de conocimiento?

Para no inferir conclusiones contradictorias hay dos grandes opciones:[34]

a) mantener la base **consistente** y usar una inferencia **monotónica** como Modus Ponens;
b) mantener la base **con potenciales contradicciones** y usar una inferencia (no monotónica) que permita **decidir** ante una contradicción.

Exigir consistencia es elegir la opción (a).

Ventajas de exigir consistencia.

- Alcanza con una **inferencia monotónica estándar** como Modus Ponens.[35]
- Se evita la **explosión**: en lógica proposicional, de un conjunto inconsistente se puede derivar todo el lenguaje ("¿aprobé? sí", "¿desaprobé? sí").[36]
- Se cumple la propiedad deseable de que el mecanismo de inferencia no concluya a la vez una cosa y exactamente lo contrario.[37]

Desventajas.

- **La información real suele ser contradictoria**: al obtener información puede haber contradicciones en el tiempo o desde distintos puntos de vista.[38] Por ejemplo, un agente puede recibir información de varios informantes,[39] o los expertos pueden dar reglas opuestas (en Mer-val, `riesgosa(X)` si está en fusión y `~riesgosa(X)` si la fusión es con una empresa fuerte).[40]
- **Cada dato nuevo obliga a revisar la base**: si lo que llega contradice lo que ya se cree, hay que **decidir qué eliminar** para recuperar la consistencia (¿lo nuevo, lo viejo, la regla?).[41] Esto requiere operadores de revisión como los de AGM.[42]
- **Se pierde información**: para garantizar la consistencia, la revisión puede eliminar creencias que estaban en la base (revisión priorizada) o rechazar la información nueva (no priorizada).[43] Lo eliminado ya no puede usarse como razón, ni a favor ni en contra.

Ventajas de exigir consistencia	Desventajas
inferencia monotónica estándar (Modus Ponens)	la información real (tiempo, informantes, expertos) suele ser contradictoria
no hay explosión (no se deriva todo el lenguaje)	cada dato nuevo obliga a decidir qué eliminar (revisión)
nunca se concluye algo y su contrario	se pierden creencias o se rechaza información nueva

[34] Cátedra, Episodio IX (valores de certeza), «Bases de creencias inconsistentes: dos grandes opciones». Misma diapositiva en Episodio I, parte 2.
[35] Cátedra, Episodio I, parte 2, «Bases de creencias inconsistentes: dos grandes opciones».
[36] Cátedra, Episodio I, parte 2, «Ejemplo: ¿Dónde está el profesor?».
[37] Cátedra, Episodio VIII, «¿Cuál conclusión debería prevalecer?».
[38] Cátedra, Episodio VIII, «Cuestiones que hay que abordar en SBC».
[39] Cátedra, Episodio IX (informantes), «Múltiples informantes con diferente credibilidad».
[40] Cátedra, Episodio VIII, «Extendiendo el mecanismo de inferencia».
[41] Cátedra, Episodio I, parte 2, «Ejemplo: ¿Dónde está el profesor?».
[42] Cátedra, Episodio I, parte 2 (dinámica de creencias), «Dinámica de creencias».
[43] Cátedra, Episodio I, parte 2 (dinámica de creencias), «Revisiones priorizadas y no priorizadas».

Consideraciones si se permite la contradicción (opción b).

- **Representar la contradicción**: hace falta un operador de **negación fuerte** (`~`), que Prolog estándar no tiene (ver ejercicio 3), y la noción de complemento (`~c` es el complemento de `c`, y viceversa).[44]
- **Extender el mecanismo de inferencia** para que no concluya a la vez `c` y `~c`. Puede hacerlo de dos formas:[45]
  1. **cauta**: no concluir ninguna de las dos. Las respuestas pasan a ser *sí*, *no*, *indeciso* (deriva las dos) o *desconozco* (no deriva ninguna);[46]
  2. **decidir cuál prevalece**, comparando la calidad de la información que sustenta cada derivación.[47]
- **Cómo comparar**: hay varias formas.
  - valores de certeza numéricos (enfoque cuantitativo);[48]
  - un orden de credibilidad entre informantes (enfoque cualitativo);[49]
  - argumentación (DeLP).
- **Tener en cuenta todas las razones**: para dar una respuesta coherente y consistente hay que considerar todos los argumentos, a favor y en contra.[50] Un conflicto también puede aparecer en un **punto intermedio** de una derivación (`~riesgosa(acme)` ataca la derivación de `~comprar_acciones(acme)`).[51]

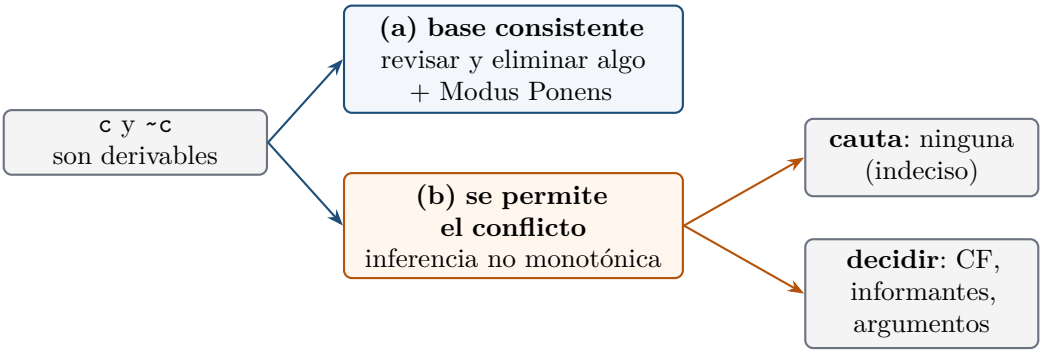


Figura 5: Qué hacer ante conclusiones contradictorias.

EXTRAS

- En la opción (b) la cátedra combina las dos ideas: en DeLP se asume que el conjunto de **hechos y reglas estrictas** (Π) no es contradictorio. Solo las **reglas rebatibles** (Δ) pueden llevar a conclusiones contradictorias.[52]
- **Monotonía**: con Modus Ponens, si una creencia se infiere de un conjunto S, también se infiere de cualquier conjunto que contenga a S.[53] El razonamiento de sentido común, en cambio, es no monotónico: "saca conclusiones" que después revisa con información más específica (como que las personas tienen dos piernas, salvo Long John Silver).[54]

[44] Cátedra, Episodio VIII, «Extendiendo el mecanismo de inferencia».
[45] Cátedra, Episodio VIII, «¿Cuál conclusión debería prevalecer?».
[46] Cátedra, Episodio VIII, «Extendiendo el mecanismo de inferencia».
[47] Cátedra, Episodio VIII, «Eligiendo la mejor conclusión para recomendar».
[48] Cátedra, Episodio IX (valores de certeza), «Conocimiento con valores de certeza (certain factor)».
[49] Cátedra, Episodio IX (informantes), «Múltiples informantes con diferente credibilidad».
[50] Cátedra, Episodio X, parte 1, «Conclusiones aceptadas o rechazadas».
[51] Cátedra, Episodio X, parte 1, «Conflicto en puntos internos a una derivación».
[52] Cátedra, Episodio X, parte 2, «Conjunto de reglas contradictorio».
[53] Cátedra, Episodio I, parte 2, «Buscando herramientas para implementar un agente deliberativo».
[54] AIMA, §12.6, p. 458–459.

- **Principio de mínimo cambio**: ante un cambio se busca preservar tanta información como sea posible. Eso descarta soluciones como eliminar todo para sacar una sola creencia.[55]
- **El costo de mantener la consistencia**: al retractar P no alcanza con borrarlo. Hay que revisar todo lo que se infirió de P, salvo lo que tenga otra justificación. Los **TMS** se encargan de eso:[56]
  - el más simple vuelve al estado anterior a P y rehace todo lo agregado después, lo que es impráctico si se agregan muchos hechos;
  - el **JTMS** anota cada sentencia con sus justificaciones y borra solo las que dependen de P en todas ellas.

---

[55] Cátedra, Episodio I, parte 2 (dinámica de creencias), «Dinámica de creencias».
[56] AIMA, §12.6.2, p. 460–461.

Ejercicio 7

Consigna

Considere la siguiente base de conocimiento que se muestra en la tabla, donde las reglas y los hechos tienen asociado un valor de certeza (enfoque cuantitativo). Utilizando la estrategia MYCIN para propagar valores numéricos considerando la base de conocimiento anterior, ¿quiénes (turuleca, donald y petete) son capaces de cruzar el lago? Justifique su respuesta comparando los valores de certeza obtenidos para cada una de las derivaciones de los literales que necesita para responder a esta pregunta.

h1	1	`gallina(turuleca) ← true`
h2	-0.3	`gallina(petete) ← true`
h3	-1	`gallina(donald) ← true`
h4	1	`pato(donald) ← true`
h5	0.5	`ave(petete) ← true`

r1	1	`ave(X) ← pato(X)`
r2	1	`ave(X) ← gallina(X)`
r3	0.8	`anida_suelo(X) ← gallina(X)`
r4	0.9	`anida_suelo(X) ← pato(X)`
r5	0.7	`~vuela(X) ← anida_suelo(X), ave(X)`
r6	0.8	`~cruza_lago(X) ← ~vuela(X), ave(X)`
r7	0.9	`cruza_lago(X) ← vuela(X)`
r8	0.7	`vuela(X) ← ave(X)`

(Los nombres h1–h5 y r1–r8 no están en el enunciado: se agregan para referirse a cada elemento.)

Respuesta: con la estrategia MYCIN, **los tres** (turuleca, petete y donald) cruzan el lago: para cada uno, el valor de certeza (vc) de `cruza_lago` es mayor que el de `~cruza_lago`. El paso a paso:

Paso 1: cómo se propagan los valores de certeza (estrategia MYCIN).

Qué se calcula	Cómo
hecho	su vc en la KB
conjunción $P_1, \ldots, P_N$	$\min(vc(P_1), \ldots, vc(P_N))$[57]
conclusión Q de la regla $Q \leftarrow P$	$vc(P) \cdot vc(Q \leftarrow P)$ si $vc(P)$ es positivo; 0 si $vc(P)$ es negativo o cero
literal con varias derivaciones	el **máximo** de los vc de sus derivaciones[58]
respuesta a la consulta Q	**sí** si hay derivación para Q y no para su complemento, o hay para ambos y $vc(Q) > vc(\overline{Q})$; **no** en el caso simétrico; **indeciso** si $vc(Q) = vc(\overline{Q})$; **desconocido** si no hay derivación para ninguno[59]

Paso 2: qué literales hacen falta.

- Para responder "¿`cruza_lago(X)`?" hay que comparar vc(`cruza_lago(X)`) con vc(`~cruza_lago(X)`).
- `cruza_lago(X)` sale de r7, que necesita `vuela(X)`; este sale de r8, que necesita `ave(X)`.
- `~cruza_lago(X)` sale de r6, que necesita `~vuela(X)` y `ave(X)`; `~vuela(X)` sale de r5, que necesita `anida_suelo(X)` y `ave(X)`.
- `ave(X)` y `anida_suelo(X)` salen de los hechos `gallina(X)` y `pato(X)` (r1–r4), y en el caso de petete también del hecho h5.
- Hay una derivación distinta por cada forma de elegir con qué regla o hecho se obtiene cada literal: si `ave` tiene 2 derivaciones y `anida_suelo` tiene 2, `~cruza_lago` tiene $2 \times 2 = 4$.

[57] Cátedra, Episodio IX (valores de certeza), «Computando el valor de certeza de las conclusiones».
[58] Cátedra, Episodio IX (valores de certeza), «Resolución del conflicto con información numérica».
[59] Cátedra, Episodio IX (valores de certeza), «¿Qué respuesta dar?».

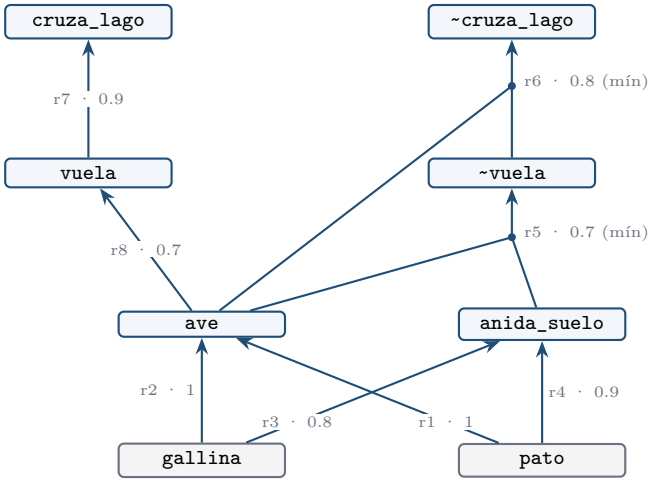


Figura 6: Dependencias entre los literales de la KB (para un `X` cualquiera). Cada flecha es una regla, con su valor de certeza; en r5 y r6 los dos antecedentes se juntan en un punto porque se combinan con el mínimo.

A continuación, para cada individuo, se siguen los pasos del ejemplo resuelto de la cátedra:[60]

1. cada derivación posible de `cruza_lago` y de `~cruza_lago`: a la izquierda, sus reglas y hechos con su valor de certeza y, en el recuadro, el *vc* de la conclusión en esa derivación; a la derecha, el cálculo;
2. para cada literal, el *vc* más alto entre sus derivaciones;
3. cada derivación como un árbol: en celeste, los valores de certeza según aparecen en la KB; en rosa, los computados con MYCIN;
4. la conclusión que elige el SBC.

Dentro de cada derivación se omite el argumento (es siempre el mismo individuo): `ave` es `ave(turuleca)`, etc.

Paso 3: turuleca.

Veamos las derivaciones posibles para `cruza_lago(turuleca)`...

```
cruza_lago ← vuela [0.9]                    0.63
vuela ← ave [0.7]
ave ← gallina [1]
gallina [1]
```

```
vc(cruza_lago) = (0.9) * vc(vuela)
vc(cruza_lago) = (0.9) * (0.7) = 0.63
vc(vuela) = (0.7) * vc(ave)
vc(vuela) = (0.7) * (1) = 0.7
vc(ave) = (1) * vc(gallina)
vc(ave) = (1) * (1) = 1
vc(gallina) = 1
```

Veamos ahora las derivaciones posibles para `~cruza_lago(turuleca)`...

[60] Cátedra, Ejemplo SBC, «Enfoque cuantitativo: Factores de certeza numéricos».

```
~cruza_lago ← ~vuela, ave [0.8]                    0.448
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← gallina [0.8]
ave ← gallina [1]
gallina [1]
```

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0.56, 1)
vc(~cruza_lago) = (0.8) * (0.56) = 0.448

vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0.8, 1)
vc(~vuela) = (0.7) * (0.8) = 0.56

vc(anida_suelo) = (0.8) * vc(gallina)
vc(anida_suelo) = (0.8) * (1) = 0.8
vc(ave) = (1) * vc(gallina)
vc(ave) = (1) * (1) = 1
vc(gallina) = 1
```

Cada literal tiene una sola derivación: $vc(\texttt{cruza_lago}) = \mathbf{0.63}$ y $vc(\texttt{\~{}cruza_lago}) = \mathbf{0.448}$.

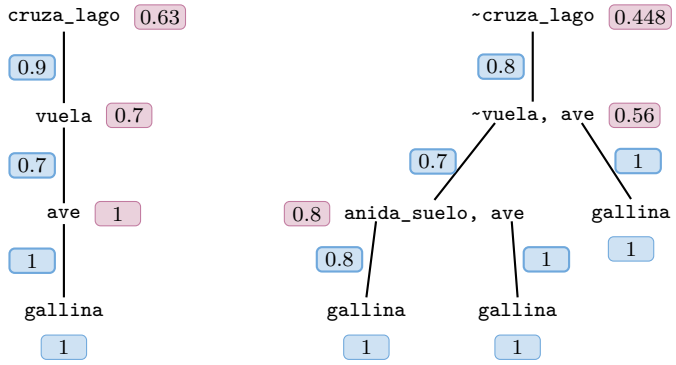


Figura 7: Las dos derivaciones de turuleca como árboles. Celeste: valores de la KB; rosa: valores computados con MYCIN.

En base a los valores de certeza, el SBC elige la conclusión `cruza_lago(turuleca)`: turuleca cruza el lago.

Paso 4: petete. Hechos: $vc(\texttt{gallina}) = -0.3$ (h2: hay algo más de certeza de que **no** es gallina) y $vc(\texttt{ave}) = 0.5$ (h5). No hay `pato(petete)`, así que r1 y r4 no se pueden usar. Como `ave` se obtiene de dos formas (h5 o r2), cada literal tiene 2 derivaciones.

Veamos las derivaciones posibles para `cruza_lago(petete)`...

```
cruza_lago ← vuela [0.9]                           0.315
vuela ← ave [0.7]
ave [0.5]
```

```
vc(cruza_lago) = (0.9) * vc(vuela)
vc(cruza_lago) = (0.9) * (0.35) = 0.315

vc(vuela) = (0.7) * vc(ave)
vc(vuela) = (0.7) * (0.5) = 0.35
vc(ave) = 0.5
```

```
cruza_lago ← vuela [0.9]                           0
vuela ← ave [0.7]
ave ← gallina [1]
gallina [-0.3]
```

```
vc(cruza_lago) = (0.9) * vc(vuela)
vc(cruza_lago) = (0.9) * 0 = 0

vc(vuela) = (0.7) * vc(ave)
vc(vuela) = (0.7) * 0 = 0
vc(ave) = 0, porque vc(gallina) ≤ 0
vc(gallina) = -0.3
```

Si tenemos más de una derivación posible para un literal, nos quedamos con el valor de certeza más alto. En este caso, $vc(\texttt{cruza_lago}) = \mathbf{0.315}$.

Veamos ahora las derivaciones posibles para `~cruza_lago(petete)`...

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← gallina [0.8]
gallina [-0.3]
ave [0.5]
```

0

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0, 0.5)
vc(~cruza_lago) = (0.8) * 0 = 0

vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0, 0.5)
vc(~vuela) = (0.7) * 0 = 0

vc(anida_suelo) = 0, porque vc(gallina) ≤ 0
vc(gallina) = -0.3
vc(ave) = 0.5
```

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← gallina [0.8]
ave ← gallina [1]
gallina [-0.3]
```

0

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0, 0)
vc(~cruza_lago) = (0.8) * 0 = 0

vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0, 0)
vc(~vuela) = (0.7) * 0 = 0

vc(anida_suelo) = 0, porque vc(gallina) ≤ 0
vc(ave) = 0, porque vc(gallina) ≤ 0
vc(gallina) = -0.3
```

En este caso, vc(`~cruza_lago`) = **0**: ninguna derivación da más que 0, porque `anida_suelo` solo se obtiene de `gallina`, que es negativo.

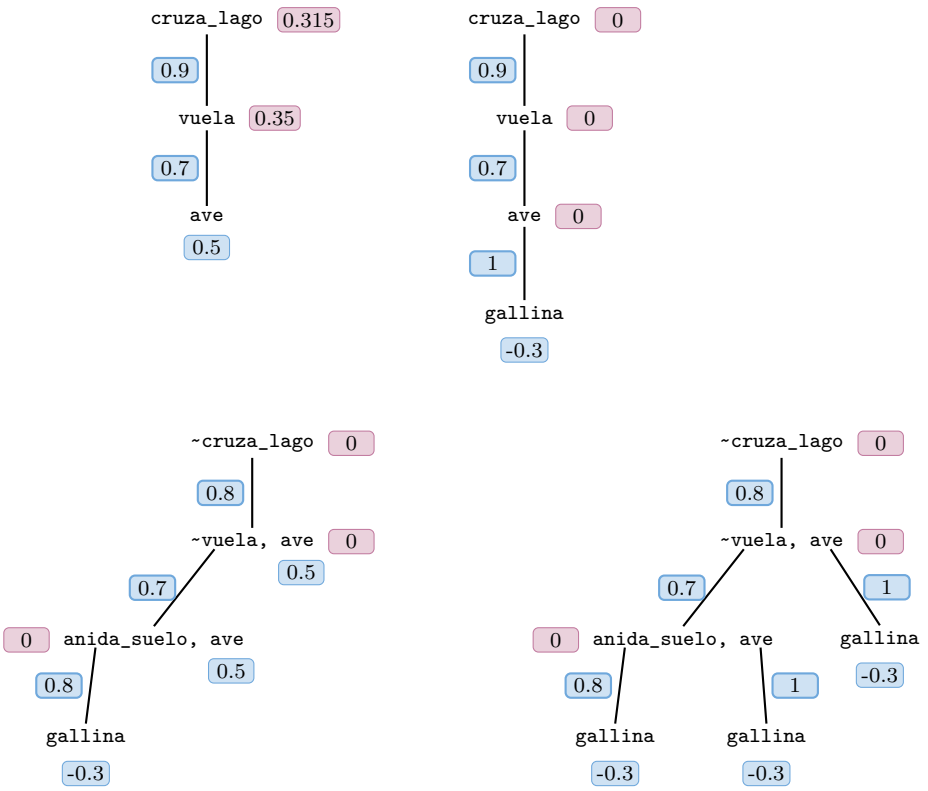


Figura 8: Las derivaciones de petete como árboles: arriba, las de `cruza_lago`; abajo, las de `~cruza_lago`. Celeste: valores de la KB; rosa: valores computados con MYCIN.

En base a los valores de certeza, el SBC elige la conclusión `cruza_lago(petete)`: petete cruza el lago.

Paso 5: donald. Hechos: $vc(\texttt{gallina}) = -1$ (h3: certeza total de que **no** es gallina) y $vc(\texttt{pato}) = 1$ (h4). `ave` se obtiene de dos formas (r1 o r2) y `anida_suelo` también (r4 o r3): `cruza_lago` tiene 2 derivaciones y `~cruza_lago`, $2 \times 2 = 4$.

Veamos las derivaciones posibles para `cruza_lago(donald)`...

```
cruza_lago ← vuela [0.9]
vuela ← ave [0.7]
ave ← pato [1]
pato [1]
```

0.63

```
vc(cruza_lago) = (0.9) * vc(vuela)
vc(cruza_lago) = (0.9) * (0.7) = 0.63
vc(vuela) = (0.7) * vc(ave)
vc(vuela) = (0.7) * (1) = 0.7
vc(ave) = (1) * vc(pato)
vc(ave) = (1) * (1) = 1
vc(pato) = 1
```

```
cruza_lago ← vuela [0.9]
vuela ← ave [0.7]
ave ← gallina [1]
gallina [-1]
```

0

```
vc(cruza_lago) = (0.9) * vc(vuela)
vc(cruza_lago) = (0.9) * 0 = 0
vc(vuela) = (0.7) * vc(ave)
vc(vuela) = (0.7) * 0 = 0
vc(ave) = 0, porque vc(gallina) ≤ 0
vc(gallina) = -1
```

Si tenemos más de una derivación posible para un literal, nos quedamos con el valor de certeza más alto. En este caso, $vc(\texttt{cruza_lago}) = \mathbf{0.63}$.

Veamos ahora las derivaciones posibles para `~cruza_lago(donald)`...

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← pato [0.9]
ave ← pato [1]
pato [1]
```

0.504

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0.63, 1)
vc(~cruza_lago) = (0.8) * (0.63) = 0.504
vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0.9, 1)
vc(~vuela) = (0.7) * (0.9) = 0.63
vc(anida_suelo) = (0.9) * vc(pato)
vc(anida_suelo) = (0.9) * (1) = 0.9
vc(ave) = (1) * vc(pato)
vc(ave) = (1) * (1) = 1
vc(pato) = 1
```

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← pato [0.9]
ave ← gallina [1]
pato [1]
gallina [-1]
```

0

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0, 0)
vc(~cruza_lago) = (0.8) * 0 = 0
vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0.9, 0)
vc(~vuela) = (0.7) * 0 = 0
vc(anida_suelo) = (0.9) * vc(pato)
vc(anida_suelo) = (0.9) * (1) = 0.9
vc(ave) = 0, porque vc(gallina) ≤ 0
vc(pato) = 1;  vc(gallina) = -1
```

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← gallina [0.8]
ave ← pato [1]
gallina [-1]
pato [1]
```

0

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0, 1)
vc(~cruza_lago) = (0.8) * 0 = 0

vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0, 1)
vc(~vuela) = (0.7) * 0 = 0

vc(anida_suelo) = 0, porque vc(gallina) ≤ 0
vc(ave) = (1) * vc(pato)
vc(ave) = (1) * (1) = 1
vc(gallina) = -1;  vc(pato) = 1
```

```
~cruza_lago ← ~vuela, ave [0.8]
~vuela ← anida_suelo, ave [0.7]
anida_suelo ← gallina [0.8]
ave ← gallina [1]
gallina [-1]
```

0

```
vc(~cruza_lago) = (0.8) * min(vc(~vuela), vc(ave))
vc(~cruza_lago) = (0.8) * min(0, 0)
vc(~cruza_lago) = (0.8) * 0 = 0

vc(~vuela) = (0.7) * min(vc(anida_suelo), vc(ave))
vc(~vuela) = (0.7) * min(0, 0)
vc(~vuela) = (0.7) * 0 = 0

vc(anida_suelo) = 0, porque vc(gallina) ≤ 0
vc(ave) = 0, porque vc(gallina) ≤ 0
vc(gallina) = -1
```

En este caso, $vc($`~cruza_lago`$) = \mathbf{0.504}$: es la única derivación que no pasa por `gallina`.

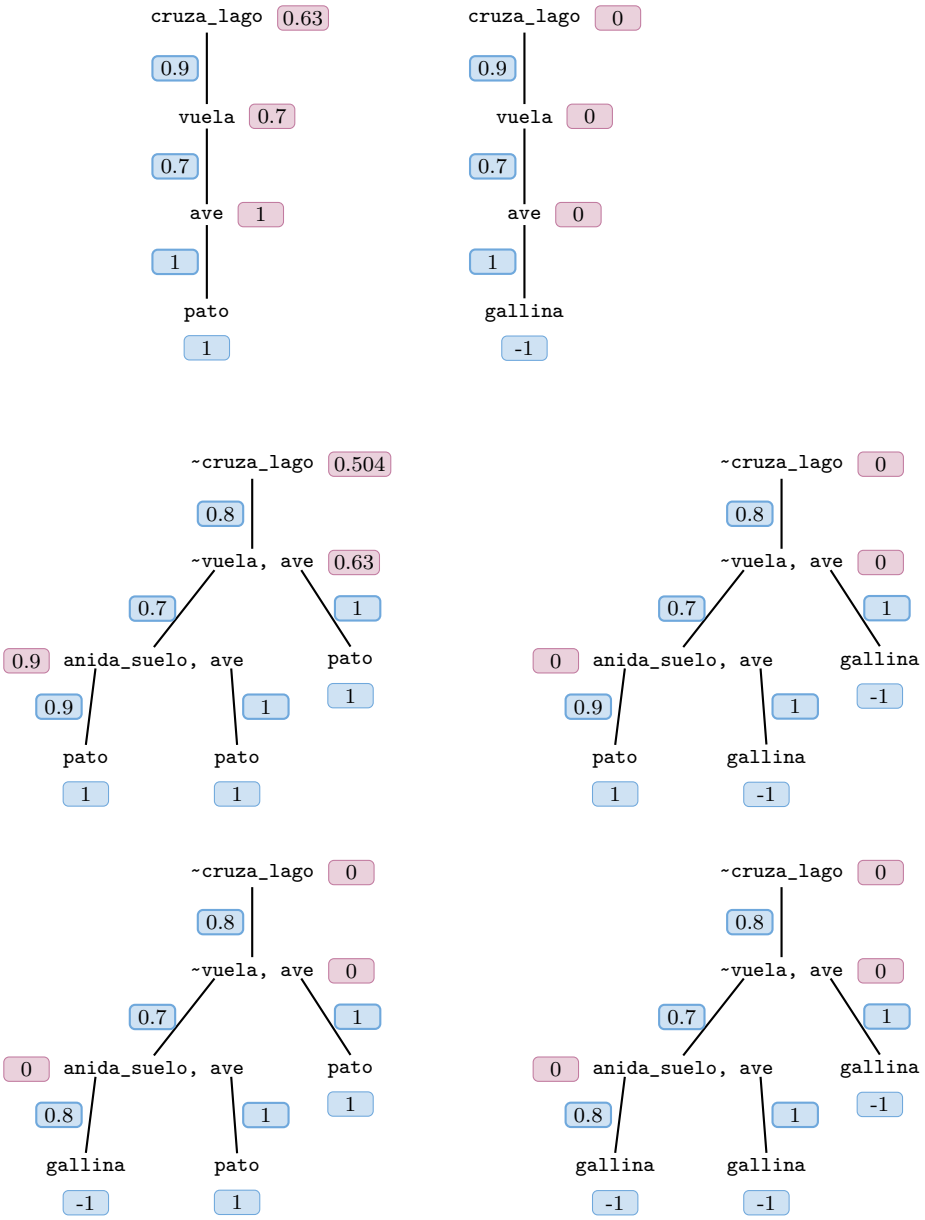


Figura 9: Las derivaciones de donald como árboles: arriba, las de `cruza_lago`; abajo, las cuatro de `~cruza_lago`. Celeste: valores de la KB; rosa: valores computados con MYCIN.

En base a los valores de certeza, el SBC elige la conclusión `cruza_lago(donald)`: donald cruza el lago.

Paso 6: comparación final.

	`vuela`	`~vuela`	`cruza_lago`	`~cruza_lago`	**¿cruza el lago?**
turuleca	0.7	0.56	**0.63**	0.448	sí
petete	0.35	0	**0.315**	0	sí
donald	0.7	0.63	**0.63**	0.504	sí

- En los tres casos, $vc($`cruza_lago`$) > vc($`~cruza_lago`$)$: la respuesta a "¿`cruza_lago(X)`?" es **sí**.
- También el conflicto intermedio `vuela` / `~vuela` se resuelve a favor de `vuela` en los tres, así que es coherente con la respuesta.

Supuestos

- **Derivaciones con vc 0**: se cuentan como derivaciones, como hace la cátedra en sus ejemplos (`~a ← e` con $vc = 0$ en el Ejemplo SBC). Si no se contaran, petete no tendría ninguna derivación para `~cruza_lago` y la respuesta sería igual: sí.
- **Hechos negativos**: en el Ejemplo SBC la cátedra escribe directamente `vc(f) = 0` para el hecho `f [-0.3]`. Acá se deja el valor del hecho y se aplica la regla "si el cuerpo es negativo o cero, la conclusión vale 0". El resultado es el mismo.[61]
- **Un literal repetido se deriva igual**: `ave` aparece dos veces en las derivaciones de `~cruza_lago` (en r5 y en r6); se toma la misma derivación de `ave` en los dos lugares (cada derivación es un conjunto de reglas y hechos, como en el ejemplo de la cátedra). Si se mezclaran (r1 en un lugar y r2 en el otro), aparecerían derivaciones extra para donald, todas con vc 0: no cambian el máximo.
- **Conflictos intermedios**: el vc de cada derivación se calcula sin mirar si un literal intermedio tiene complemento derivable (así lo hace la cátedra en sus ejemplos). Acá no cambia nada, porque `vuela` gana sobre `~vuela` para los tres.

EXTRAS

- **La propagación es una elección**: hay muchas maneras de computar el CF de una conclusión; la de la cátedra es una forma muy simple basada en MYCIN. Al implementar un sistema se puede usar otra que se adapte mejor al dominio.[62]
- **Ventaja y desventaja de los CF**: las computadoras hacen cómputos numéricos con facilidad y hay mucho sustento matemático. La desventaja es decidir qué valor asociar a cada sentencia: ¿de dónde surge?, ¿quién lo decide?, ¿0.7 o 0.71?[63]
  - Ejemplo con esta KB: si r7 tuviera $vc = 0.7$ en vez de 0.9, $vc($`cruza_lago(donald)`$) = 0.7 \cdot 0.7 = 0.49 < 0.504$ y donald **no** cruzaría; turuleca sí ($0.49 > 0.448$).
- **MYCIN** diagnosticaba infecciones de la sangre con unas 450 reglas, obtenidas entrevistando expertos. Los CF fueron su forma de reflejar la incertidumbre del conocimiento médico.[64]
- **Los CF son *truth-functional***: el valor de una sentencia compuesta se calcula solo a partir del de sus componentes (como acá, con el mínimo y el producto), y la propagación es local.[65] Según AIMA, eso funciona solo con reglas puramente diagnósticas o causales y poco interconectadas:[66]
  - "*In other circumstances, certainty factors could yield disastrously incorrect degrees of belief through overcounting of evidence.*"
  - al crecer la KB había que retocar (*tweak*) los CF de otras reglas al agregar una nueva; por eso las redes bayesianas los reemplazaron.

[61] Cátedra, Ejemplo SBC, «Enfoque cuantitativo: Factores de certeza numéricos»; regla en Episodio IX (valores de certeza), «Computando el valor de certeza de las conclusiones».
[62] Cátedra, Episodio IX (valores de certeza), «Computando el valor de certeza de las conclusiones».
[63] Cátedra, Episodio IX (valores de certeza), «Uso de valores de certeza en un SBC».
[64] AIMA, §1.3.5, p. 23.
[65] AIMA, §14.7.1, p. 547–548.
[66] AIMA, §14.7.1, p. 549.

Ejercicio 8

Consigna

Considere la siguiente base de conocimiento, donde las reglas y los hechos tienen asociado un informante (enfoque cualitativo).

h1	Ar	`camino_abierto`	r1	At	`~camino_abierto ← nieve`
h2	Ap	`camino_abierto`	r2	Av	`~viaje ← ~camino_abierto`
h3	Ai	`~nieve`	r3	Av	`viaje ← camino_abierto`
h4	Ap	`~nieve`	r4	Ai	`camino_abierto ← ~nieve`
h5	Ai	`nieve`			

(El enunciado da los elementos en otro orden y sin nombres; h1–h5 y r1–r4 se agregan para referirse a cada uno.)

a. Considerando el siguiente orden entre los informantes: Av < Ar < Ap < At < Ai < Ac, y asumiendo el criterio de comparación entre derivaciones "enlace más débil", indique cuál sería la decisión tomada por un agente con respecto a viajar. Justifique su respuesta comparando el informante asociado a cada una de las derivaciones de los literales que necesita para responder a esta pregunta.

b. Indique un orden de informantes para que la decisión del inciso anterior cambie.

Inciso a. **Respuesta**: el agente queda **indeciso** (no decide ni viajar ni no viajar): la mejor derivación de `viaje` y la de `~viaje` tienen el mismo informante, **Av**. Todas pasan por una regla de Av (r3 o r2), y Av es el informante menos creíble. El paso a paso:

Paso 1: cómo se compara con informantes.

Qué se calcula	Cómo
confianza de una derivación	el informante **menos creíble** (mínimo) de sus elementos: "eslabón más débil"[67]
literal con varias derivaciones	el **máximo** de las confianzas de sus derivaciones[68]
respuesta a la consulta Q	**sí** si hay derivación para Q y no para su complemento, o hay para ambos y confianza(Q) > confianza($\overline{Q}$); **no** en el caso simétrico; **indeciso** si las confianzas son iguales; **desconocido** si no hay derivación para ninguno[69]

Paso 2: qué literales hacen falta.

- Para decidir si viajar hay que comparar la confianza de `viaje` con la de `~viaje`.
- `viaje` sale de r3, que necesita `camino_abierto`; este sale de h1, de h2 o de r4 (que necesita `~nieve`, hecho h3 o h4). Son **4 derivaciones**.
- `~viaje` sale de r2, que necesita `~camino_abierto`; este sale de r1, que necesita `nieve` (h5). Es **1 derivación**.

Se siguen los pasos del ejemplo resuelto de la cátedra:[70]

1. cada derivación posible de `viaje` y de `~viaje`: a la izquierda, sus reglas y hechos con su informante y, en el recuadro, el informante de la conclusión en esa derivación; a la derecha, el cálculo;
2. para cada literal, el mejor informante entre sus derivaciones;
3. cada derivación como un árbol: en celeste, los informantes según aparecen en la KB; en rosa, los obtenidos con el eslabón más débil;

[67] Cátedra, Episodio IX (informantes), «Resolución del conflicto con información simbólica».
[68] Cátedra, Episodio IX (informantes), «Resolución del conflicto basado en informantes».
[69] Cátedra, Episodio IX (informantes), «¿Qué respuesta dar?».
[70] Cátedra, Ejemplo SBC, «Enfoque cualitativo: Orden de credibilidad entre informantes».

4. la conclusión que elige el SBC.

Av < Ar < Ap < At < Ai < Ac

Veamos las derivaciones posibles para `viaje`...

```
viaje ← camino_abierto [Av]
camino_abierto [Ar]
```
Av

```
conf(viaje) = mín({Av, Ar}) = Av
```

```
viaje ← camino_abierto [Av]
camino_abierto [Ap]
```
Av

```
conf(viaje) = mín({Av, Ap}) = Av
```

```
viaje ← camino_abierto [Av]
camino_abierto ← ~nieve [Ai]
~nieve [Ai]
```
Av

```
conf(viaje) = mín({Av, Ai, Ai}) = Av
conf(camino_abierto) = mín({Ai, Ai}) = Ai
```

```
viaje ← camino_abierto [Av]
camino_abierto ← ~nieve [Ai]
~nieve [Ap]
```
Av

```
conf(viaje) = mín({Av, Ai, Ap}) = Av
conf(camino_abierto) = mín({Ai, Ap}) = Ap
```

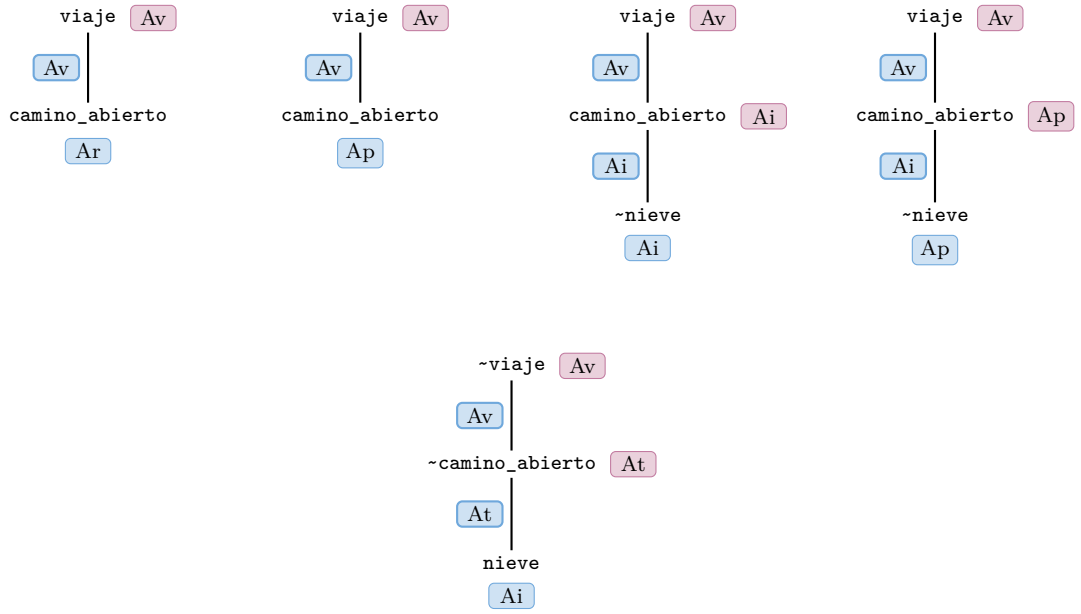
Si tenemos más de una derivación posible para un literal, nos quedamos con el mejor informante posible asociado. En este caso, las cuatro dan Av: confianza(`viaje`) = **Av**.

Y las derivaciones posibles para `~viaje`...

```
~viaje ← ~camino_abierto [Av]
~camino_abierto ← nieve [At]
nieve [Ai]
```
Av

```
conf(~viaje) = mín({Av, At, Ai}) = Av
conf(~camino_abierto) = mín({At, Ai}) = At
```

Hay una sola derivación: confianza(`~viaje`) = **Av**.



$$\text{Av} < \text{Ar} < \text{Ap} < \text{At} < \text{Ai} < \text{Ac}$$

Figura 10: Las derivaciones del inciso a como árboles: arriba, las cuatro de `viaje`; abajo, la de `~viaje`. Celeste: informantes de la KB; rosa: informantes obtenidos con el eslabón más débil.

confianza(`viaje`) = confianza(`~viaje`) = Av: el SBC no puede preferir ninguna de las dos conclusiones. Respuesta: indeciso. El agente no tiene una decisión sobre viajar.

Inciso b. **Respuesta**: con el orden **At < Av < Ar < Ap < Ai < Ac** (At pasa de la mitad al último lugar; el resto queda igual), el agente **viaja**.

- Las derivaciones de `viaje` no usan At: siguen dando Av.
- La de `~viaje` sí usa At (r1), que ahora es el eslabón más débil: da At.
- Como Av > At, se prefiere `viaje`.

$$\text{At} < \text{Av} < \text{Ar} < \text{Ap} < \text{Ai} < \text{Ac}$$

Veamos las derivaciones posibles para `viaje`...

```
viaje ← camino_abierto [Av]
camino_abierto [Ar]
```
Av

```
conf(viaje) = mín({Av, Ar}) = Av
```

```
viaje ← camino_abierto [Av]
camino_abierto [Ap]
```
Av

```
conf(viaje) = mín({Av, Ap}) = Av
```

```
viaje ← camino_abierto [Av]
camino_abierto ← ~nieve [Ai]
~nieve [Ai]
```
Av

```
conf(viaje) = mín({Av, Ai, Ai}) = Av
conf(camino_abierto) = mín({Ai, Ai}) = Ai
```

```
viaje ← camino_abierto [Av]
camino_abierto ← ~nieve [Ai]
~nieve [Ap]
```
Av

```
conf(viaje) = mín({Av, Ai, Ap}) = Av
conf(camino_abierto) = mín({Ai, Ap}) = Ap
```

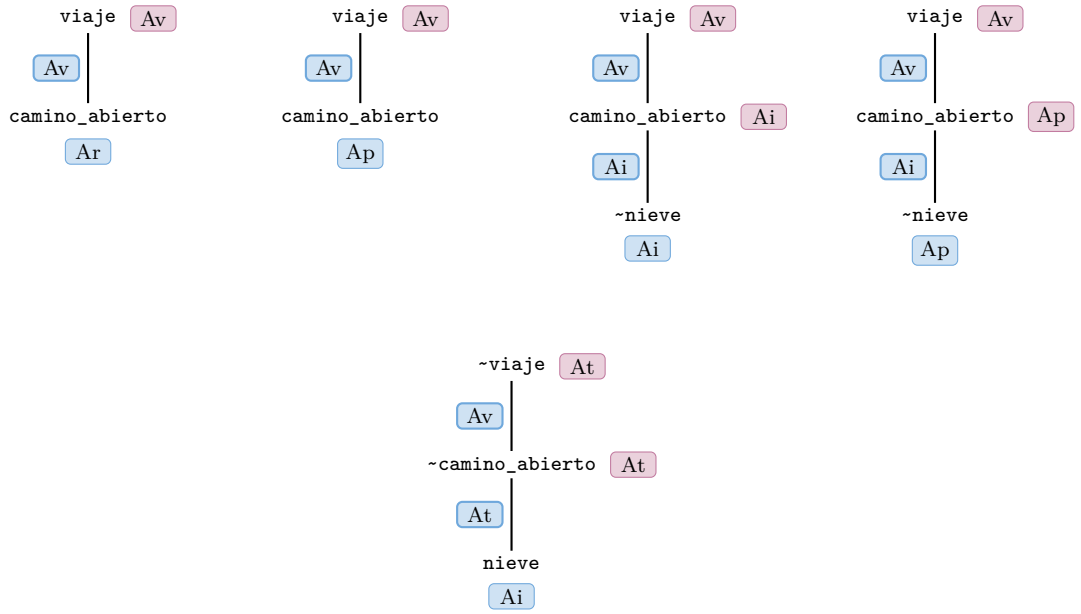
confianza(`viaje`) = **Av**.

Y las derivaciones posibles para `~viaje`...

```
~viaje ← ~camino_abierto [Av]
~camino_abierto ← nieve [At]
nieve [Ai]
```
At

```
conf(~viaje) = mín({Av, At, Ai}) = At
conf(~camino_abierto) = mín({At, Ai}) = At
```

confianza(`~viaje`) = **At**.



At < Av < Ar < Ap < Ai < Ac

Figura 11: Las derivaciones del inciso b como árboles: arriba, las cuatro de `viaje`; abajo, la de `~viaje`. Celeste: informantes de la KB; rosa: informantes obtenidos con el eslabón más débil.

En base a los informantes, el SBC elige la conclusión `viaje` (Av > At): el agente viaja.

Supuestos

- **Ac** aparece en el orden pero no etiqueta ningún elemento de la KB; no interviene en ninguna derivación.
- **Conflicto intermedio**: `camino_abierto` y `~camino_abierto` también se derivan los dos. Como en el ejercicio 7, la confianza de cada derivación se calcula sin mirar ese conflicto (así lo hace la cátedra en sus ejemplos). Igual se resuelve a favor de `camino_abierto` en los dos incisos: Ai contra At.

EXTRAS

- **Cambiar el orden sin tocar la KB**: si el agente descubre un cambio en la credibilidad de sus fuentes, puede cambiar el orden sin reasignar las etiquetas de la KB, y con eso puede preferirse otra conclusión.[71] Es lo que pasa en el inciso b.

[71] Cátedra, Episodio IX (informantes), «El orden de credibilidad puede cambiar dinámicamente».

- **Con esta KB, ningún orden total hace que el agente decida no viajar**:
  - la derivación de `~viaje` usa Av, At y Ai; la tercera de `viaje` (r3, r4, h3) usa solo Av y Ai;
  - el mínimo de un subconjunto nunca es menor que el del conjunto, así que confianza(`viaje`) $\geq$ confianza(`~viaje`) con cualquier orden: la respuesta es **sí** o **indeciso**, nunca **no**.

  Para que gane `~viaje` habría que cambiar los informantes de la KB (por ejemplo, r3 de Av a Ar, con Ar como el menos creíble: `viaje` daría Ar y `~viaje`, Av) o agregar reglas.

Ejercicio 9

Consigna

Considere la base de conocimiento que se muestra en la siguiente tabla. Para cada uno de los incisos, muestre todas las derivaciones para los literales a, ~a, b y ~b. Acompañe cada derivación con el valor de certeza o el informante asociado, según corresponda.

`a ← b, c`	`a ← d`	`a ← ~b`	`b`
`~a ← c`	`~a ← b, d`	`~a ← e`	`c`
`b ← c`	`~b ← d`	`~b ← c`	`d`

a)

Consigna

Utilizando la estrategia MYCIN, asigne valores de certeza para cada uno de los elementos de la base de conocimiento, de forma tal que se cumplan las siguientes restricciones:

I. El literal a tiene que ser preferido a ~a.
II. Alguna derivación para ~a debe ser mejor que alguna derivación para a.
III. No puede existir una regla con cabeza a y valor de certeza 1.
IV. No se debe poder preferir a b sobre ~b, ni viceversa.

Respuesta: valores de certeza asignados:

vc	**Elemento de la KB**
0.9	`a ← b, c`
0.9	`a ← d`
0.5	`a ← ~b`
0.7	`~a ← c`
1	`~a ← b, d`
−0.8	`~a ← e`
1	`b ← c`
1	`~b ← d`
1	`~b ← c`
−0.1	`b`
0.5	`c`
−0.3	`d`

- Los *vc* se propagan con la estrategia MYCIN, como en el ejercicio 7 (paso 1).
- Las derivaciones, como árboles: en celeste, los valores de la KB; en rosa, los computados con MYCIN.
- `~a ← e` no da ninguna derivación: `e` no es un hecho ni la cabeza de una regla.

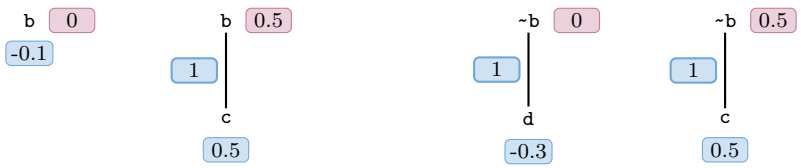


Figura 12: Derivaciones de `b` (izquierda) y de `~b` (derecha). Celeste: valores de la KB; rosa: valores computados con MYCIN.

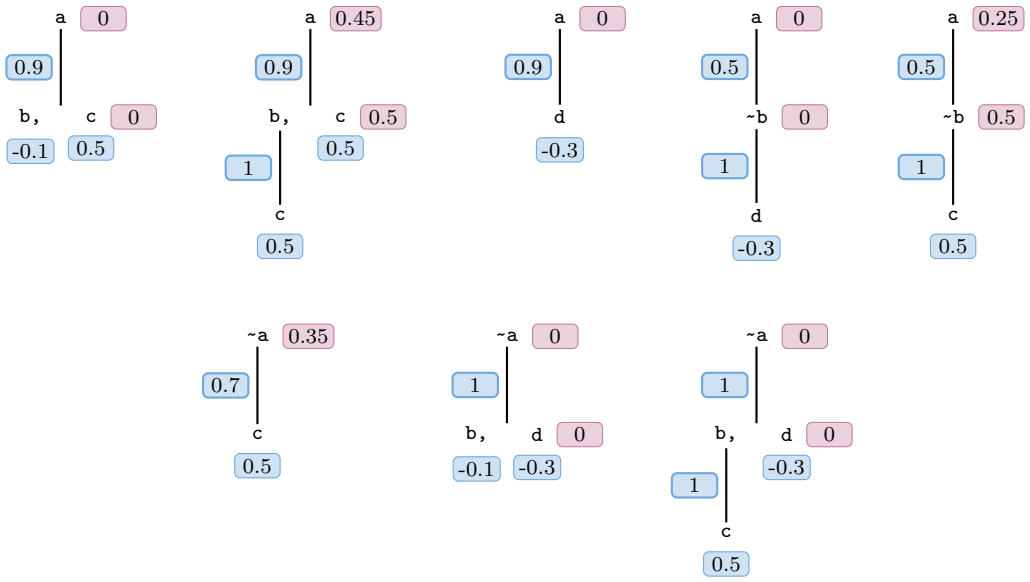


Figura 13: Derivaciones de `a` (arriba) y de `~a` (abajo). Celeste: valores de la KB; rosa: valores computados con MYCIN (en un cuerpo de dos literales, el rosa es el mínimo de los dos).

Tomando para cada literal el máximo entre sus derivaciones y respondiendo a cada consulta según los *vc*:[72]

Literal	**Derivaciones**	*vc*	**Respuesta**
`a`	5	**0.45**	sí
`~a`	3	0.35	no
`b`	2	0.5	indeciso
`~b`	2	0.5	indeciso

Restricción	**Con estos valores**	
i.	$vc(\texttt{a}) = 0.45 > vc(\texttt{\~a}) = 0.35$	✓
ii.	la derivación `~a ← c` (0.35) es mejor que `a ← ~b`, `~b ← c` (0.25)	✓
iii.	las reglas con cabeza `a` tienen *vc* 0.9, 0.9 y 0.5	✓
iv.	$vc(\texttt{b}) = vc(\texttt{\~b}) = 0.5$	✓

Supuestos

- Las derivaciones con *vc* 0 se cuentan, como en el ejercicio 7. Si no se contaran, ii se cumple igual: la comparación usa la derivación de 0.25, no las de 0.

EXTRAS

- **Cómo se eligieron los valores**:
  - `c` es el único hecho positivo: toda derivación con $vc > 0$ pasa por `c`. Los hechos negativos (`b`, `d`) anulan las derivaciones que los usan, así que `a ← d`, `~a ← b, d` y `~b ← d` quedan en 0.
  - iv: `b ← c` y `~b ← c` tienen el mismo cuerpo, así que con el mismo *vc* en las dos reglas (acá 1) empatan.
  - i: `a ← b, c` y `~a ← c` terminan con el mismo cuerpo efectivo (0.5), y gana la regla de mayor *vc* (0.9 contra 0.7).

[72] Cátedra, Episodio IX (valores de certeza), «Resolución del conflicto con información numérica» y «¿Qué respuesta dar?».

    - ii: `a ← ~b` con *vc* bajo (0.5) da una derivación de `a` peor que la de `~a`.
- El -0.8 de `~a ← e` no influye: cualquier valor da lo mismo mientras `e` no se pueda derivar.
